Fakultas Informatika
School of Computing
Telkom University

Pemrograman Berorientasi Objek

Exception

Ex~~In~~ception

High Demand from user

- Users have high expectations for the code we produce.
- Users will use our programs in unexpected ways.
- Due to design errors or coding errors, our programs may fail in unexpected ways during execution

Errors In General

- Syntax errors
  - The rules of the language have not been followed.
  - detected by the compiler
- Runtime errors
  - Error while the program is running and the environment detects an operation that is impossible to carry out
- Logic errors
  - a program doesn't perform the way it was intended to

Error Example

```
int arr[] = {4,5,2,0,-1};

int i;

Scanner s = new Scanner(System.in);

i = s.nextInt();

System.out.println(arr[ i ]);
```

Exception

- an indication of a problem that occurs during a program's execution
- An abnormal event, which occurs during the execution of a program, that disrupts the normal flow of the program's instructions.
- Here, we call that error as Exception

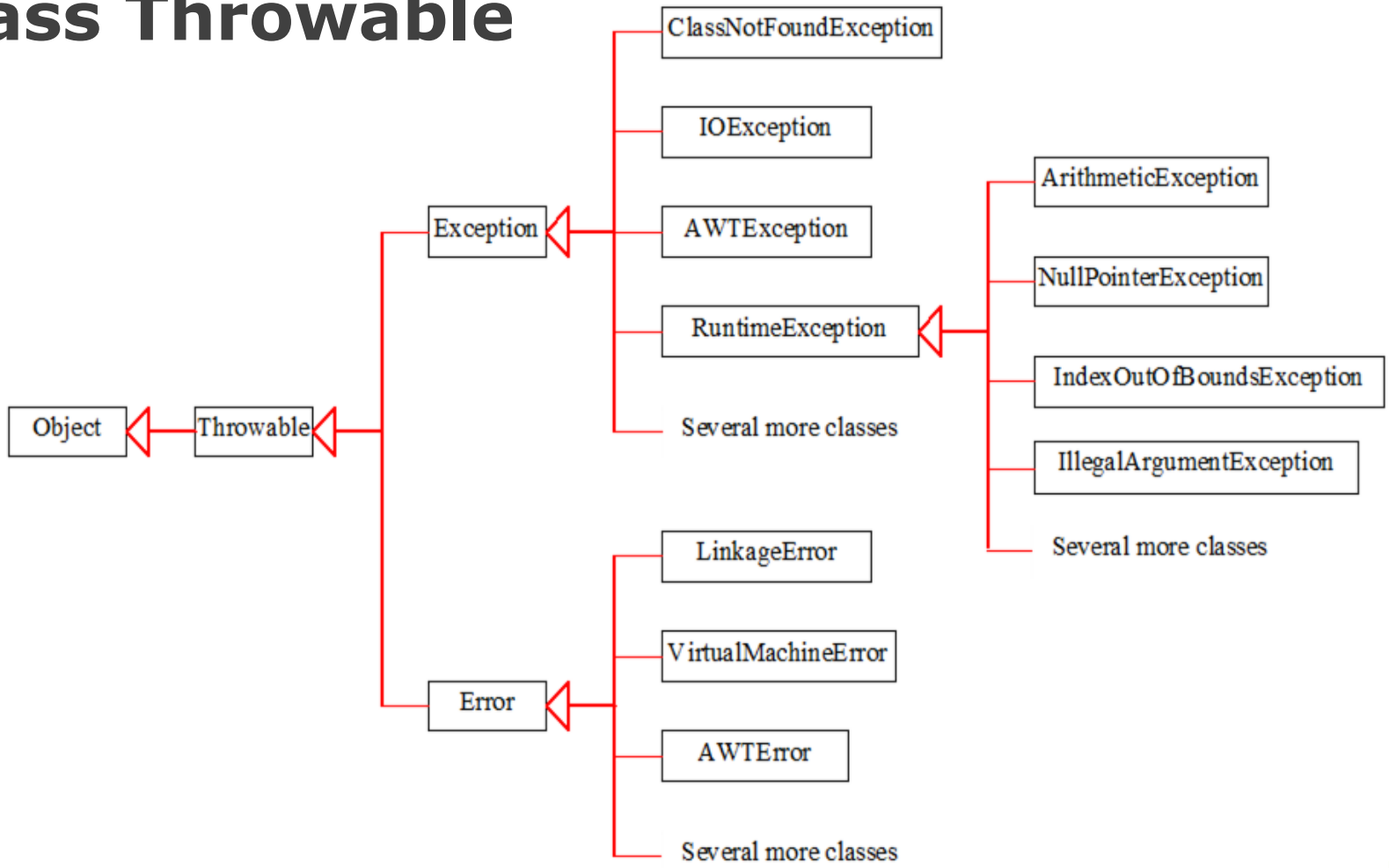
Error and Exception

- Error usually is a condition where no programmer can guess and can handle it
  - Usually called System Error
  - Let the program terminate
- Exception can be guessed and can be handled
  - Ought to Handled by programmer

Exception Handling

- process of responding to the occurrence of *exceptions* during computation
- A technique to prevent the program ended prematurely because of exception
- Technique to create a program that can resolve (or handle) exceptions

Class Throwable



Exception Handling Keyword

- Try
  - Try some code
- Catch
  - Catch the exceptions
- Throw
  - Throw the exceptions
- Throws
  - Declare the thrown exception
- Finally
  - Code to run after exception

Exception Handling

```
try {
    // code that might cause an exception
} catch ( ExceptionTipe e) {
    // code to handle the exception
} finally {
    // code to execute even if an unexpected exception occurs
}
```

Exception Example

Basic Example

```
public class TryException {
	public int number;

	public void setNumber(int number){
		this.number = number;
	}

	public int getNumber(){
		return number;
	}
}
```

```
import java.util.Scanner;

public class Driver{
	public static void main(String args[]){
		TryException t = new TryException();
		Scanner s = new Scanner(System.in);

		int number = s.nextInt();

		t.setNumber(number);
	}
}
```

What if user input some characters ?

Basic Example

```
public class TryException {
	public int number;

	public void setNumber(int number){
		this.number = number;
	}

	public int getNumber(){
		return number;
	}
}
```

```
import java.util.Scanner;

public class Driver{
	public static void main(String args[]){
		TryException t = new TryException();
		Scanner s = new Scanner(System.in);
		int number = 0;
		try{
			number = s.nextInt();
		} catch (Exception e){
			System.out.println("exception occurs");
		}

		t.setNumber(number);
	}
}
```

Surround with try-catch Block

If exception occurs, number is still = 0

Wrong Example

```
public class TryException {
	public int number;

	public void setNumber(int number){
		this.number = number;
	}

	public int getNumber(){
		return number;
	}
}
```

```
import java.util.Scanner;

public class Driver{
	public static void main(String args[]){
		TryException t = new TryException();
		Scanner s = new Scanner(System.in);

		try{
			int number = s.nextInt();
		} catch (Exception e){
			System.out.println("exception occurs");
		}

		t.setNumber(number);
	}
}
```

Error, as int number will not exists when exception occurs

Another Example

```
public class TryException {
	public int number;

	public void setNumber(int number){
		this.number = number;
	}

	public int getNumber(){
		return number;
	}
}
```

```
import java.util.Scanner;

public class Driver{
	public static void main(String args[]){
		TryException t = new TryException();
		Scanner s = new Scanner(System.in);

		try{
			int number = s.nextInt();
			t.setNumber(number);
		} catch (Exception e){
			System.out.println("exception occurs");
		}

	}
}
```

If exception occurs, t.setNumber will not be executed

Catch According to its Exception

```
public class People {
	public String name;

	public People(String name){
		this.name = name;
	}

	public String getName(){
		return name;
	}
}
```

```
import java.util.Scanner;

public class Driver{
	public static void main(String args[]){
		People list[] = new People[4];
		list[0] = new People("erick");
		list[2] = new People("adam");

		Scanner s = new Scanner(System.in);

		int id = 0;
		id = s.nextInt();
		System.out.println(list[id].getName());
	}
}
```

Several exception that might happen :

- Input not a number
- Input number > 3
- Index has not been instantiated

Catch According to its Exception

```
public class People {
	public String name;

	public People(String name){
		this.name = name;
	}

	public String getName(){
		return name;
	}
}
```

```
public static void main(String args[]){
	People list[] = new People[4];
	list[0] = new People("erick");
	list[2] = new People("adam");
	Scanner s = new Scanner(System.in);
	int id = 0;
	try{
		id = s.nextInt();
		System.out.println(list[id].getName());
	} catch (java.util.InputMismatchException e){
		System.out.println("input not a number");
	} catch (ArrayIndexOutOfBoundsException e){
		System.out.println("input > array size");
	} catch (NullPointerException e){
		System.out.println("array "+id+
				" has not been instantiated");
	} catch (Exception e){
		System.out.println("if everything else fails");
	}
}
```

Catch According to its Exception

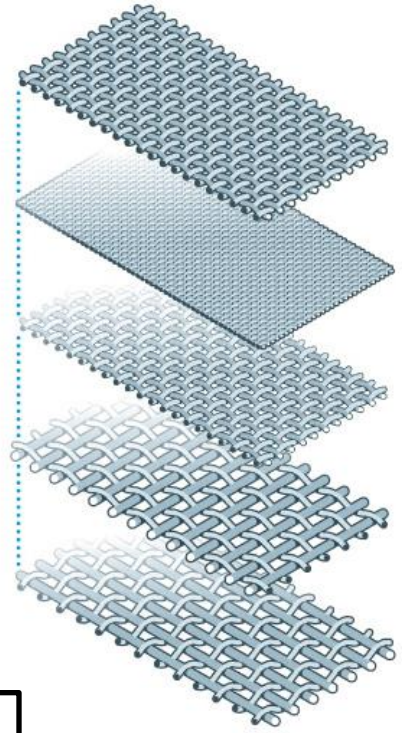
```
public static void main(String args[]){
      People list[] = new People[4];
      list[0] = new People("erick");
      list[2] = new People("adam");
      Scanner s = new Scanner(System.in);
      int id = 0;
      try{
            id = s.nextInt();
            System.out.println(list[id].getName());
      } catch (java.util.InputMismatchException e){
            System.out.println("input not a number");
      } catch (ArrayIndexOutOfBoundsException e){
            System.out.println("input > array size");
      } catch (NullPointerException e){
            System.out.println("array "+id+
                        " has not been instantiated");
      } catch (Exception e){
            System.out.println("if everything else fails");
      }
}
```

```
>  2
>> adam

>  7
>> input > array size

>  1
>> array 1 has not been
   instantiated

>  x
>> input not a number
```

Catch Exception

```
try{
        // code to try
} catch (Exception e){
        // catch exception
} catch (ArrayIndexOutOfBoundsException e){
        // catch exception
} catch (NullPointerException e){
        // catch exception
}
```

Do not put default exception as the first catch



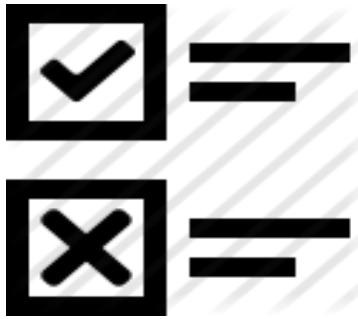
Imagine catching exception is like a layered filters,
Use default exception catch if everything else fails to catch the specific exceptions

Common Exception

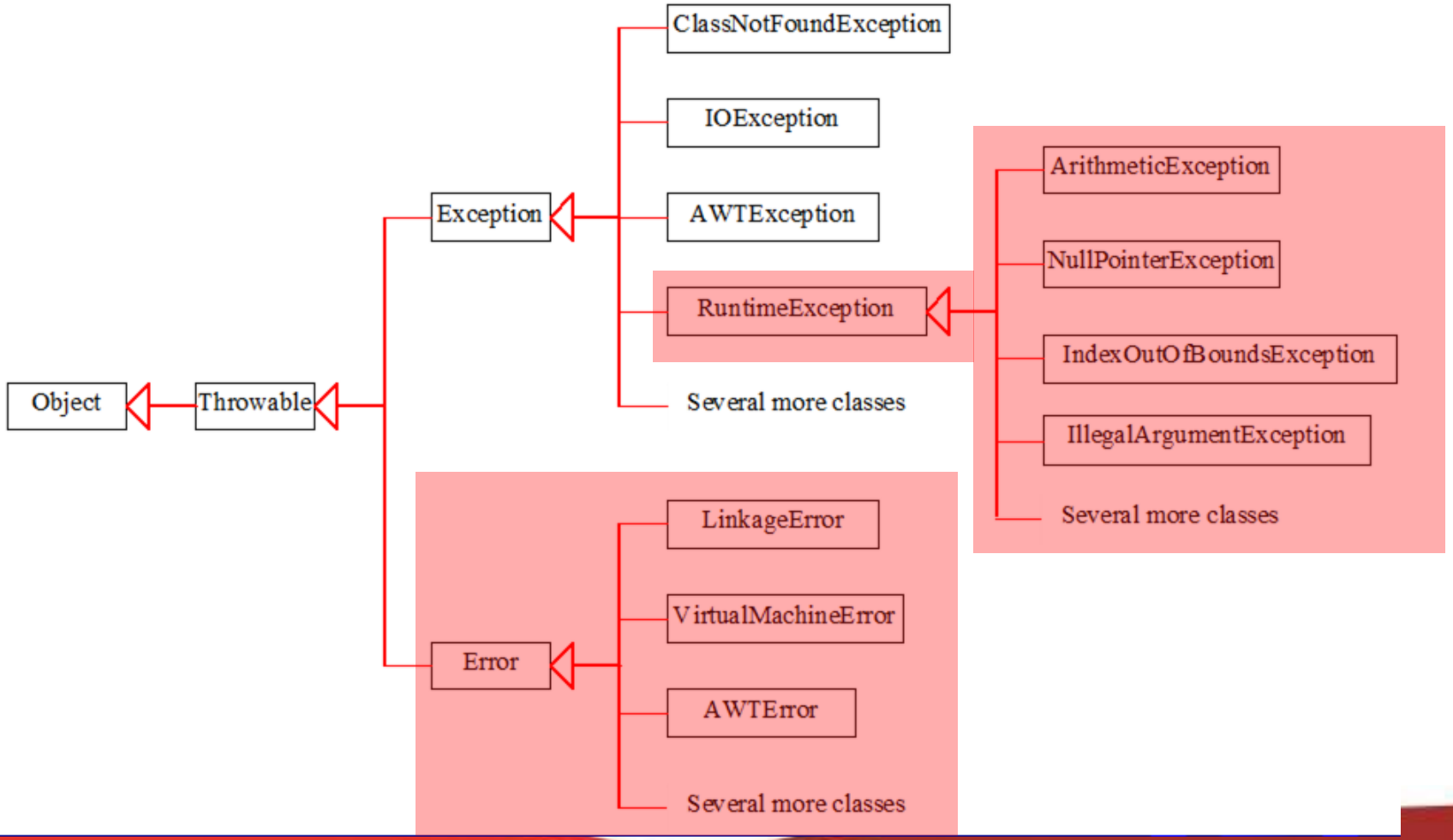
- ArithmeticException
- NullPointerException
- NegativeArraySizeException
- ArrayIndexOutOfBoundsException
- SecurityException

Checked and Unchecked Exception

- Checked Exception
  - Expected exception by JVM (compiler)
  - the compiler forces the programmer handle the exception
  - High probability of exception to happen
    - Input output exception
    - AWT exception
- Unchecked Exception
  - Exception during the runtime
  - Not verified during Compile time
  - Must be detected by programmers



Checked and Unchecked Exception



Checked Exception

```
import java.io.*;

public class Driver{

	public static void main(String[] args) {
		File test = new File("d:\\some file.txt");
		test.createNewFile();
	}

}
```

IO Operation must be checked

Checked Exception

```
import java.io.*;

public class DemoFileException {
	public static void main(String[] args) {
		try {
			File test = new File("d:\\some file.txt");
			test.createNewFile();
		}
		catch (IOException e) {
			System.out.println("IO Exception occurs");
			System.out.println(e);
		}
		System.out.println("Finished");
	}
}
```

Checked Exception

- Exception
  - IOException
  - FileNotFoundException
  - ParseException
  - ClassNotFoundException
  - CloneNotSupportedException
  - InstantiationException
  - InterruptedException
  - NoSuchMethodException
  - NoSuchFieldException

Exception In Methods

Handling Exception in a Method

```
public class TryException{
    int number[] = new int[5];

    public void setNumber(int id, int number) {
        this.number[id] = 5/number;
    }
    public int getNumber(int id){
        return number[id];
    }
}
```

We detect that these codes might cause some Exceptions when called

```
public class Driver {
    public static void main(String[] args) {

        TryException t = new TryException();

        t.setNumber(8, 10);
        System.out.println(t.getNumber(4));

    }
}
```

Handling Exception in a Method

```
public class TryException{
    int number[] = new int[5];

    public void setNumber(int id, int number) {
        this.number[id] = 5/number;
    }
    public int getNumber(int id){
        return number[id];
    }
}
```

```
public class Driver {
    public static void main(String[] args) {

        TryException t = new TryException();

        try{
            t.setNumber(8, 10);
            System.out.println(t.getNumber(4));
        } catch (Exception e){
            System.out.println("exception hoccurs");
        }
    }
}
```

can't determine which one that made the exception

Exception in setNumber, getNumber won't be executed

Handling Exception in a Method

```
public class TryException{
    int number[] = new int[5];

    public void setNumber(int id, int number) {
        try {
            this.number[id] = 5/number;
        } catch (Exception e) {
            System.out.println("error in setNumber");
        }
    }

    public int getNumber(int id) {
        try{
            return number[id];
        }catch(Exception e){
            return 0;
        }
    }
}
```

```
public class Driver {
    public static void main(String[] args) {

        TryException t = new TryException();

        t.setNumber(8, 10);
        System.out.println(t.getNumber(4));

    }
}
```

Throw the Exception

- Handle the exception on another class
  - Output in driver/application class
- Use another method when exception occurs
- Use keyword "**throw**"

Throw the Exception

```
public class TryException {
   int number[] = new int[5];
      public void setNumber(int id, int number) {
            try {
                  this.number[id] = 5/number;
            } catch (ArrayIndexOutOfBoundsException e) {
                  throw new ArrayIndexOutOfBoundsException ("error in setNumber")
            }
      }
      public int getNumber(int id) {
            try{
                  return number[id];
            }catch(Exception e){
                  return 0;
            }
      }
}
```

Throw the Exception

```
public class Driver {
	public static void main(String[] args) {

		TryException t = new TryException();

		try {
			t.setNumber(7, 5);
		} catch (Exception e) {
			System.out.println(e.getMessage());
		}

		System.out.println(t.getNumber(8));

	}
}
```

```
> Error in set Number
> 0
```

Multiple Throws

```
public class TryException {
   int number[] = new int[5];

      public void setNumber(int id, int number) {
            try {
                  this.number[id] = 5/number;
            } catch (ArrayIndexOutOfBoundsException e) {
                  throw new ArrayIndexOutOfBoundsException ("index out of bounds");
            } catch (ArithmeticException e) {
                  throw new ArithmeticException ("error division by zero");
            }
      }

      ...
```

Custom Throw

```
public class TryException {
   int number[] = new int[5];

      public void setNumber(int id, int number) {
            if ( id >= this.number.length || id < 0 ) {
                  throw new ArrayIndexOutOfBoundsException ("index out of bounds");
            } else if ( number == 0 ) {
                  throw new ArithmeticException ("error division by zero");
            } else {
                  this.number[id] = 5/number;
            }
      }

      ...
```

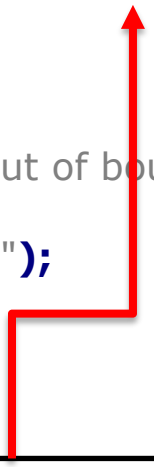
Throws Exception Example

Declaring the thrown exception

- Declare the list of every exception might thrown by the method
- Let other class know what exception may occurs
- Use keyword "**throws**"

Declaring the thrown exception

```
public class TryException {
   int number[] = new int[5];

      public void setNumber                          throws
                              ArrayIndexOutOfBoundsException, ArithmeticException {

            try {
                  this.number[id] = 5/number;
            } catch (ArrayIndexOutOfBoundsException e) {
                  throw new ArrayIndexOutOfBoundsException ("index out of bounds");
            } catch (ArithmeticException e) {
                  throw new ArithmeticException ("error division by zero");
            }

      }

      ...
```



But since these exceptions are unchecked exceptions, this is somehow useless

Declaring the thrown exception

```
public class TryException {
   int number[] = new int[5];

      public void setNumber(int id, int number) throws IOException  {

            try {
                  this.number[id] = 5/number;
            } catch (ArrayIndexOutOfBoundsException e) {
                  throw new ArrayIndexOutOfBoundsException ("index out of bounds");
            } catch (ArithmeticException e) {
                  throw new ArithmeticException ("error division by zero");
            } catch (IOException e) {
                  throw new IOException("error no such method");
            }

      }

      …
```

However, if there is any checked exception thrown, then the throws exception must be declared

Declaring the thrown exception

```
...
    public void setNumber(int id, int number) throws Exception{
        this.number[id] = 5/number;
    }

    public void setNumber2(int id, int number) {
        this.number[id] = 5/number;
    }
...
```

```
public class Driver {
    public static void main(String[] args) {

        TryException t = new TryException();

        t.setNumber(7, 5);    //compile error

        t.setNumber2(7, 5);  //compile ok
    }
}
```

Declaring a checked exception throws will make the method cannot be executed outside block try-catch

Declaring the thrown exception

```
public class TryException{
    int number[] = new int[5];

    public void setNumber(int id, int number) throws Exception {
        this.number[id] = 5/number;
    }

    public int getNumber(int id){
        try{
            return number[id];
        } catch(Exception e){
            return 0;
        }
```

We can use this to tell others that this method might cause some exception, so those who want to use it was forced catch the Exception

```
public class Driver {
    public static void main(String[] args) {

        TryException t = new TryException();

        try{
            t.setNumber(8, 10);
        } catch (Exception e){
            System.out.println("exception hoccurs");
        }

        System.out.println(t.getNumber(4));

    }
}
```

Finally Example

The Finally Block

- Continue the code regardless of any exception occurs
- Always executed when the try block exits
- Put a cleanup code
- Use keyword "**finally**"

The Finally Block

```
public class TryException {
   int number[] = new int[5];
      public void setNumber(int id, int number) {
            try {
                  this.number[id] = 5/number;

                  System.out.println("if exception occurs, this won't be executed");

            } catch (ArrayIndexOutOfBoundsException e) {
                  throw new ArrayIndexOutOfBoundsException ("error in setNumber");
            }

            System.out.println("neither will this");

      }
}
```

After an exception occurs, the rest of the code won't be executed

The Finally Block

```
public class TryException {
   int number[] = new int[5];
      public void setNumber(int id, int number) {
            try {
                  this.number[id] = 5/number;

                  System.out.println("if exception occurs, this won't be executed");

            } catch (ArrayIndexOutOfBoundsException e) {
                  throw new ArrayIndexOutOfBoundsException ("error in setNumber");
            } finally {

                  System.out.println("but this will");

            }
      }
}
```

Codes in finally block will be executed after block try exits

Finally Block on Custom Throw

```
int number[] = new int[5];

public void setNumber(int id, int number) {
   try {
       if ( id >= this.number.length || id < 0 ) {
           throw new ArrayIndexOutOfBoundsException ("index out of bounds");
       } else if ( number == 0 ) {
           throw new ArithmeticException ("error division by zero");
       } else
           this.number[id] = 5/number;
   } finally {
       System.out.println("code to run even exception occurs");
   }
}
```

Custom Exception Class

Custom Exception Class

- Defining new exception
- Exception need to provide additional information
  - Parsing value
  - Custom mechanism design to do when exception happened
- Extends "**Exception**" or "**Throwable**"
- Example:
  - User registration
  - Create an exception if username has already been taken

Example without Exception

```
public class User {
    private String id, name;
    public User(String id, String name) {
        this.id = id;
        this.name = name;
    }

    public String getID() {
        return id;
    }

    public String getName() {
        return name;
    }
}
```

```
import java.util.ArrayList;
public class Aplication {
    ArrayList<User> users = new ArrayList();

    public void addUser (User p) {
        boolean exists = false;
        for (User p1 : users) {
            if (p1.getID().equals(p.getID())) {
                exists = true;
            }
        }
        if (!exists) {
            users.add(p);
        }
    }
}
```

Example without Exception

```
public class Driver {
    public static void main(String[] args) {
        Aplication w1 = new Aplication();
        w1.addUser(new User("001","Erick"));
        w1.addUser(new User("002","Danny"));
        w1.addUser(new User("001","Bob"));
    }
}
```

Don’t know whether this success or not

Example with Exception

```
import java.util.ArrayList;
public class Aplication {
    ArrayList<User> users = new ArrayList();

    public void addUser(User p) {
        for (User p1 : users) {
            if (p1.getID().equals(p.getID())) {
                throw new RuntimeException("User "+p.getID()+" Already exists");
            }
        }
        users.add(p);
    }
}
```

Example with Exception

```
public class Driver {
    public static void main(String[] args) {
        Aplication w1 = new Aplication();
        try {
            w1.addUser(new User("001", "Erick"));
            w1.addUser(new User("002", "Danny"));
            w1.addUser(new User("001", "Bob"));
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```

User 001 already exists

Custom Class Exception

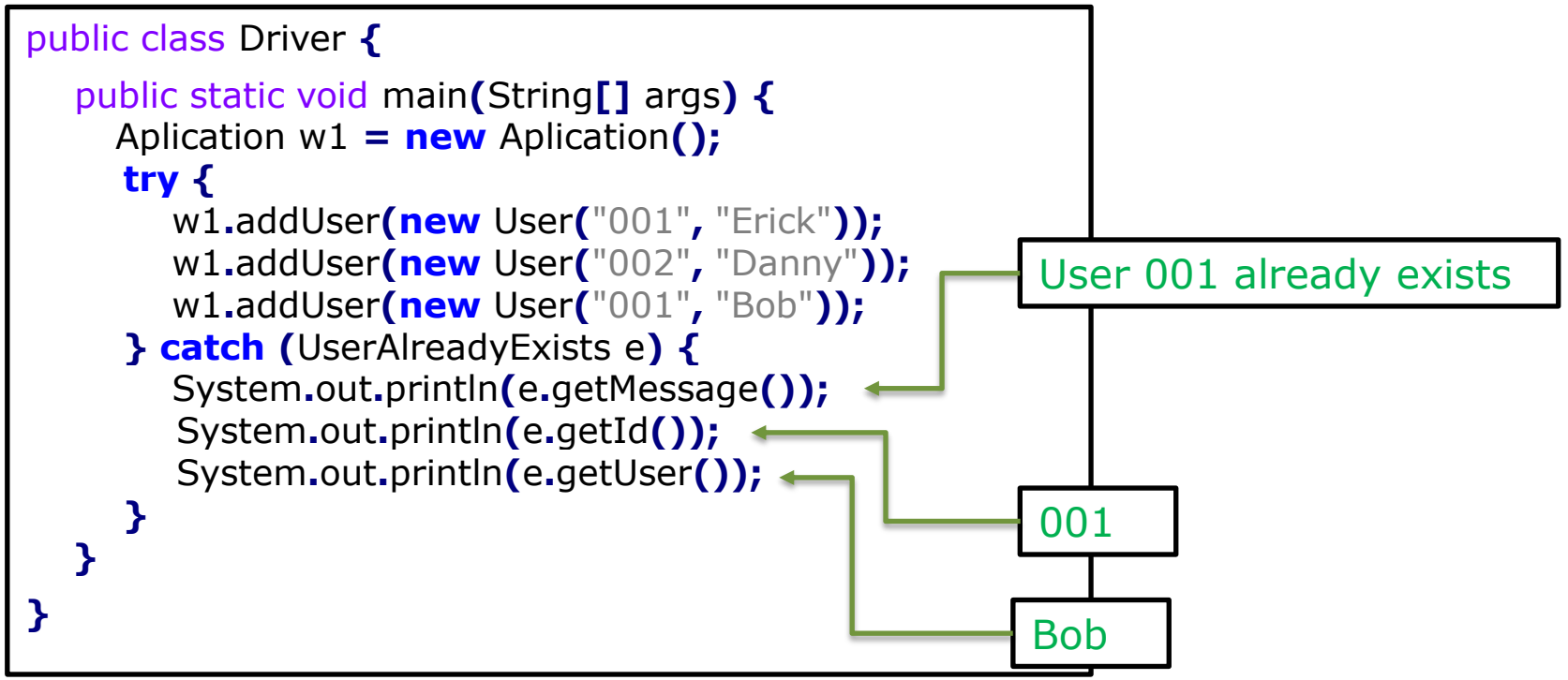
```
public class UserAlreadyExists extends RuntimeException {
 private String id;
 private String user;

    public UserAlreadyExists(String message , String id , String user) {
        super(message);
        this.user = user;
        this.id = id;
    }

 public String getUser() {
        return user;
    }

    public String getId() {
        return id;
    }
}
```

Example with Custom Class Exception

```
import java.util.ArrayList;
public class Aplication {
   ArrayList<User> users = new ArrayList();

   public void addUser(User p) {
      for (User p1 : users) {
         if (p1.getID().equals(p.getID())) {
            throw new UserAlreadyExists("User "+p.getID()+" Already exists",
                                                    p.getID(),  p.getName() );
         }
      }
      users.add(p);
   }
}
```

Example with Custom Class Exception

```
public class Driver {
    public static void main(String[] args) {
        Aplication w1 = new Aplication();
        try {
            w1.addUser(new User("001", "Erick"));
            w1.addUser(new User("002", "Danny"));
            w1.addUser(new User("001", "Bob"));
        } catch (UserAlreadyExists e) {
            System.out.println(e.getMessage());
            System.out.println(e.getId());
            System.out.println(e.getUser());
        }
    }
}
```



Exception Best Practices

- Don’t use Exception to control application behavior.
  - Exception handling is very expensive as it require native calls to copy stacktrace each time exception is created
- While creating custom exception, prefer to create an unchecked, Runtime exception than a checked exception,
  - especially if you know that client is not going to take any reactive action other than logging

Exception Best Practices

- Don't make a Exception class as nested class even if its used only by one class,
  - Always declare Exceptions in their own class
- Always provide meaning full message on Exception
- Avoid overusing Checked Exception
- Document any Exception thrown by any method

Good to read

- http://javarevisited.blogspot.sg/2013/03/0-exception-handling-best-practices-in-Java-Programming.html
- http://www.tutorialspoint.com/java/java_exceptions.htm

Question?

Java Assertion

Java Assertion

- a statement in the Java™ programming language that enables you to test your assumptions about your program
- Each assertion contains a boolean expression that you believe will be true when the assertion executes.
- If it is not true, the system will throw an error.

Java Assertion

- By verifying that the boolean expression is indeed true, the assertion confirms your assumptions about the behavior of your program, increasing your confidence that the program is free of errors.
- writing assertions while programming is one of the quickest and most effective ways to detect and correct bugs

Things you must know

- pre-conditions (in private methods only)
  - the requirements which a method requires its caller to fulfill
- post-conditions
  - verify the promises made by a method to its caller
- class invariants
  - validate object state
- unreachable-at-runtime code
  - parts of your program which you expect to be unreachable, but which cannot be verified as such at compile-time (often else clauses and default cases in switch statements)

Using Assertion

- `assert [condition];`
- `assert [condition] : [message exception] ;`

- When the list of condition return false, the assertion will throw an Assertion Error with the message defined

Assertion Example

```
import java.util.Scanner;
public class AssertionTest1 {
   public static void main(String args[]) {
      Scanner reader = new Scanner(System.in);
      System.out.print("Enter your age: ");
      int age = reader.nextInt();

      assert age>=18 : "You are too young to vote";
      System.out.println("You are eligible to vote");
   }
}
```

```
> Enter your age: 20
>> You are eligible to vote
```

```
> Enter your age: 8
>> Exception in thread "main"
java.lang.AssertionError: You are
too young to vote
```

Assertion Example

```
public static void main(String argv[]) {
    Scanner reader = new Scanner(System.in);
    System.out.print("Enter your gender [m/f]: ");
    char gender = reader.next().charAt(0);
    switch (gender) {
        case 'm':
        case 'M': System.out.println("Male"); break;
        case 'f':
        case 'F': System.out.println("Female"); break;
        default: assert !true : "Invalid Option"; break;
    }
}
```

```
> Enter your gender [m/f]: m
>> Male
```

```
> Enter your gender [m/f]: x
>> Exception in thread "main"
java.lang.AssertionError: Invalid
Option
```

Use Assertion to find bug

- For example, we create a code for something like this
  - you might have written something to explain your assumption

```
if (i % 3 == 0) {
    System.out.println("mod = 0");
} else if (i % 3 == 1) {
    System.out.println("mod = 1");
} else { // We know (i % 3 == 2)
    System.out.println("mod = 2");

}
```

Use Assertion to find bug

- You should now **use an assertion whenever you would have written a comment that asserts an invariant**

```
if (i % 3 == 0) {
    System.out.println("mod = 0");
} else if (i % 3 == 1) {
    System.out.println("mod = 1");
} else { // We know (i % 3 == 2)
    assert i % 3 == 2 : i;
    System.out.println("mod = 2");
}
```

Note, incidentally, that the assertion in the above example may fail if i is negative, as the % operator is not a true modulus operator, but computes the remainder, which may be negative

Why Assertion?

- Let's assume that you are supposed to write a program to control a nuclear power-plant.
  - It is pretty obvious that even the most minor mistake could have catastrophic results, therefore your code has to be bug-free
  - assuming that the JVM is bug-free for the sake of the argument

Why Assertion?

- Java is not a verifiable language
  - you cannot calculate that the result of your operation will be perfect.
- The main reason for this are pointers:
  - they can point anywhere or nowhere,
  - therefore they cannot be calculated to be of this exact value,
  - at least not within a reasonable span of code

Why Assertion?

- Given this problem, there is no way to prove that your code is correct at a whole.
  - But what you can do is to prove that you at least find every bug when it happens

Design by Contract

- based on the DbC paradigm:
  - you first define (with mathematically precision) what your method is supposed to do,
  - and then verify this by testing it during actual execution.

Example

```
// Calculates the sum of a (int) + b (int)
// and returns the result (int).
int sum(int a, int b) {
  return a + b;
}
```

- While this is pretty obvious to work fine, most programmers will not see the hidden bug inside this one
  - (hint: the Ariane V crashed because of a similar bug).
- Now the DbC defines that you must always check the input and output of a function to verify that it did work correct

Example

```
// Calculates the sum of a (int) + b (int)
// and returns the result (int).
int sum(int a, int b) {
    assert (Integer.MAX_VALUE - a >= b) :
        "Value of " + a + " + " +b+ " is too large to add.";
  final int result = a + b;
    assert (result - a == b) :
        "Sum of " + a + " + " +b+ " returned wrong sum " + result;
  return result;
}
```

- Should this function now ever fail, you will notice it.
  - You will know that there is a problem in your code,
  - you know where it is and you know what caused it (similar to Exceptions)

And what is even more important:

- you stop executing right when it happens
- to prevent any further code to work with wrong values and potentially cause damage to whatever it controls.

Java Exceptions are a similar concept

- but they fail to verify everything.
- If you want even more checks (at the cost of execution speed) you need to use assertions.
- Doing so will bloat your code, but you can in the end deliver a product at a surprisingly short development time (the earlier you fix a bug, the lower the cost).

And in addition

- if there is any bug inside your code, you will detect it.
- There is no way of a bug slipping-through and cause issues later.
- This still is not a guarantee for bug-free code,
  - but it is much closer to that, than usual programs.

Careful with Assertion

- Do not use assertions to check the parameters of a public method.
- Do not use assertions to do any work that your application requires for correct operation.

Question?

Fakultas Informatika
School of Computing
Telkom University

THANK YOU